

ControlSci Data Agent 部署指南

版本: v4.1 | 更新日期: 2026-05-24

适用环境: Windows 10/11 + Conda + Python 3.12+ | 可选: Ollama (本地模型)、vLLM WSL2 (1.2B VLM 文档解析)、RTX 5090 24GB (QLoRA 微调 + 35B 推理)

架构: 15 intents × LLM Intent Router × 四路径资源调度 (API/MiMo-V2.5 视觉/Ollama 本地/MinerU)

1. 系统概述

ControlSci Data Agent 是面向 AGI4S 的科学文档跨模态语料智能体。它能自主理解科学文档处理需求、规划多步执行路径、调用四路径推理引擎 (API DeepSeek/MiMo/MiniMax 含 MiMo-V2.5 原生视觉、Ollama 本地 9B/35B、Ollama Vision、vLLM WSL2 MinerU-1.2B VLM 公式识别) + MinerU 文档解析, 完成从原始 PDF 到结构化跨模态评测数据集的全链路自主生产。

1.1 核心能力

能力	说明
15 intents × LLM Intent Router	自然语言 → few-shot 拆解为多步 plan → 自动调度执行路径 → Verify 闭环; 新增 <code>sciverse_search</code> 作为跨源文献检索 intent
四路径推理引擎	API (DeepSeek/MiMo/MiniMax)、Ollama 本地 (9B/35B)、Ollama Vision、vLLM WSL2 (MinerU-1.2B VLM 文档解析) → ResourceScheduler 按任务类型自动路由
跨模态对齐审计	MiMo-V2.5 原生全模态视觉模型 (~4.7s/图), 6,204 含图 chunk × 11,554 张图片全量扫描, 断点续跑
三 Provider 并行生成	DeepSeek / MiMo / MiniMax 三路同时生成评测题
数据飞轮闭环	arxiv_search → mineru_parse → benchmark_build → quality_arbitrate → model_evaluate → leaderboard_viz: 从论文检索到排行榜更新的全自动 6-step
多格式文档解析	支持 PDF/PPTX/DOCX/XLSX/PNG/扫描件图片统一解析, chunk 数/公式提取率/图片保留率三维度对比; HTML 不作为当前 MinerU 路径承诺
LLM-as-Judge 仲裁	两层仲裁 (Embedding 快速通道 + LLM 深度仲裁) 筛选高质量题目
隐私双路径	<code>--local</code> 模式: Resource Scheduler 降级至 Ollama (Intent Router 仍用 DeepSeek)
多 Judge 一致性	API 8 Judge + 本地 6 Judge → Fleiss' κ 三角验证 + 评分者反规模定律
QLoRA 领域微调	单卡 24GB 极限微调, Perplexity 探针验证语料信息密度
结构化执行日志	JSON 格式记录每步输入/输出/状态/耗时/错误/验证结果

1.2 系统架构

```
1  用户自然语言输入: "审计这批 arXiv 论文的公式-图片跨模态对齐质量"
2  |
3  ▼
4  └─ Layer 1: Intent Router (agent_cli.py - LLM few-shot)
5  |   DeepSeek v4-flash 将自然语言 → intent 序列:
6  |   [cross_modal_audit → corpus_quality_report]
7  |   每步标注: tool / resource_type / depends_on
8  |
9  └─ Layer 2: Resource Scheduler (resource_scheduler.py)
10 |   四路径决策 (优先级: 可复现性 > 隐私 > 成本):
11 |   └─ API → DeepSeek/MiMo/MiniMax - 文本/Judge + MiMo-V2.5 视觉
12 |   └─ Ollama → qwen3.5:9b/35b - 本地推理 + Judge
13 |   └─ Ollama Vision → qwen3.5:9b - 医学图片描述与视觉检索增强
14 |   └─ vLLM WSL2 → MinerU-1.2B VLM - 公式识别专用
15 |   可选 --local: Layer 2 降级至 Ollama/MinerU (Layer 1 Intent Router 仍用 DeepSeek)
16 |
17 └─ Layer 3: Executor + Verifier
18 |   for each step:
19 |     execute → capture LogStep → verify (LLM)
20 |     └─ ✓ pass → next step
21 |     └─ ✗ fail → auto-fallback (switch provider / retry)
22 |     └─ ⚠ partial → append correction step
23 |
24 └─ Output: execution_log.json + artifacts
```

1.3 15 intents 概览

#	intent_id	功能	工具链	资源类型
0	arxiv_search	arXiv 论文检索 — 控制科学前沿论文检索+下载	searching-arxiv-papers Skill + arXiv API	script
1	mineru_parse	文档解析 — PDF/PPTX 原始文档 → 结构化 Markdown	mineru-to-md Skill + MinerU API	local_api
2	corpus_parse	语料解析 — 科学文档批量解析, 含公式/图片/表格提取	MinerU API + mineru-to-md Skill	local_api
3	cross_modal_audit	跨模态对齐审计 — 图片×公式语义一致性	visual_audit.py + MiMo-V2.5 原生视觉	api
4	corpus_quality_report	语料质量报告 — 全维度质量分析	DeepSeek + 统计脚本	api
5	benchmark_build	评测数据集构建 — 500 题四维度覆盖	build_benchmark.py + 三 Provider	api
6	quality_arbitrate	质量仲裁 — 双层 LLM 一致性筛选	arbiter.py + DeepSeek + Ollama	api
7	model_evaluate	模型评测 — 统一接口评测任意模型	evaluate.py + 自动选 Judge	api
8	multi_judge_compare	多 Judge 对比 — API vs 本地双榜	evaluate.py (API + Ollama 双路径)	api

#	intent_id	功能	工具链	资源类型
9	leaderboard_viz	排行榜可视化 — 表格+柱状图+热力图	summarize_multi.py + report_viz.py	script
10	local_finetune	本地微调 — QLoRA + Perplexity 验证	train_qlora.py + Ollama	local_gpu
11	multi_format_parse	多格式文档解析 — PPTX/DOCX/XLSX/PNG/扫描件 → Markdown 质量对比	MinerU API + tools/mineru_to_md.py	local_api
12	medical_rag	临床证据自动化合成 — 医学文献切片、索引、检索与 evidence synthesis	controlsci.medical + FAISS/BM25 + Ollama	local_only
13	sciverse_search	Sciverse 文献检索 — 跨学科正式出版物语义检索，与 arXiv 预印本检索互补	Sciverse agentic-search API + source selection	api
14	reproduce_all	全量复现 — 一键端到端全链路	run_agent.ps1 全链路	script

完整注册表见 `benchmark/agent/agent_capabilities.json`，其中 `meta.total_intents=15`。补充可靠性验收见 `docs/submissions/data_trace_bundle/12_final_supplemental_experiments/track2_agent_reliability/`：router robustness、failure injection、多源选择 A/B、资源 Pareto 与 hard-document stress 均已登记到 DATA-TRACE #169-173。

2. 环境准备

2.1 硬件要求

资源	最低配置	推荐配置
内存	8 GB	16 GB+（Ollama 35B 建议 64 GB）
磁盘	10 GB（语料 + 中间产物）	20 GB+
GPU	不需要（纯 API 调用）	RTX 5090 24GB（QLoRA + Ollama 35B）
网络	稳定互联网（API 调用）	无代理直连（降低延迟）

2.2 软件依赖

组件	版本要求	用途
Python	≥ 3.10（推荐 3.12）	主运行时
Conda	任意版本	环境管理
MinerU	≥ 1.0	PDF/论文解析
Ollama	≥ 0.6	可选：本地模型推理 + Judge
OpenAI Python SDK	≥ 1.68	DeepSeek / MiMo 文本调用
Anthropic Python SDK	≥ 0.97	MiniMax 兼容调用

组件	版本要求	用途
httpx	≥ 0.28	MiMo-V2.5 视觉 API 调用

2.3 Python 包依赖

```
1 conda run --no-capture-output -n myenv pip install openai anthropic httpx tqdm requests pyarrow
```

版本参考（已验证可用）：

包	版本	用途
openai	≥ 1.68	DeepSeek / MiMo 文本 / Ollama 兼容口
anthropic	≥ 0.97.0	MiniMax Anthropic 兼容接口
httpx	≥ 0.28.1	MiMo-V2.5 视觉 raw httpx 调用
tqdm	≥ 4.67.3	进度展示
requests	≥ 2.32+	通用 HTTP
pyarrow	≥ 15.0+	HF 数据集导出

2.4 API 密钥配置

ControlSci Data Agent 支持四类 API 密钥，需配置以下环境变量：

```
1 # DeepSeek (Intent Router + 生成 + Judge 主引擎)
2 [System.Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "<DEEPSEEK_API_KEY>", "User")
3
4 # MiMo-V2.5 视觉 + MiMo 文本 (跨模态审计 + 补充生成)
5 [System.Environment]::SetEnvironmentVariable("MIMO_API_KEY", "<MIMO_API_KEY>", "User")
6
7 # MiniMax (补充生成)
8 [System.Environment]::SetEnvironmentVariable("MINIMAX_API_KEY", "<MINIMAX_API_KEY>", "User")
9
10 # MinerU API (语料解析)
11 [System.Environment]::SetEnvironmentVariable("MINERU_API_BASE", "http://localhost:8000", "User")
```

注意：

- 环境变量设置后需重启终端生效
- DeepSeek 通过 OPENAI_API_KEY 传递（OpenAI 兼容接口）
- MiMo-V2.5 视觉调用使用 api-key header 认证，不走 OpenAI SDK（详见 §3.1 五 Provider 认证方式表）

- Agent 启动时自动执行 `_check_env()` 验证全部 Provider Key

2.5 Ollama 本地模型（轨道 2）

Ollama 提供零成本本地推理能力，支持两种通信模式：

- **原生 API** (`/api/chat`，推荐)：设置 `think:false` 避免推理链占用 `max_tokens`
- **OpenAI 兼容 API** (`/v1/chat/completions`)：非思考场景使用

```
1 # 安装 ollama 后拉取模型
2 ollama pull qwen3.5:9b          # 本地 Judge + 评测目标模型
3 ollama pull qwen3-embedding:4b  # Embedding 快速通道
4 ollama pull qwen3.5:35b        # 可选：6 Judge 一致性升级 + 弹性伸缩实验
5
6 # 验证运行状态
7 ollama list
```

Ollama 默认监听 `http://localhost:11434`。可通过 `OLLAMA_HOST` 环境变量自定义地址。

性能参考：9B 模型 ~1s/题，4B 模型 ~0.5s/题，35B 模型 ~1.5s/题（RTX 5090）。35B 是 thinking model，必须用 `/api/chat` + `think:false`，不可用 `/v1/chat/completions`。

2.6 vLLM 本地 VLM（轨道 3 — WSL2 可选）

轨道 3 通过 vLLM 在 WSL2 容器内部署 MinerU-1.2B VLM，提供专用文档解析能力。实验表明，1.2B 稠密模型在公式识别任务上的表现与 310B 通用视觉模型（MiMo-V2.5 MoE，15B 激活参数）可比——印证了“领域适配 > 模型尺寸”的工程结论。

2.6.1 前置条件

- Windows 11 + WSL2 + Ubuntu 22.04+（Windows 10 21H2+ 亦可）
- WSL2 内 Conda 环境已配置
- 宿主机 RTX 5090 24GB（vLLM 自动识别 GPU）

2.6.2 安装与启动

```
1 # WSL 内
2 conda run --no-capture-output -n myenv pip install vllm
3 vllm serve opendatalab/MinerU2.5-2509-1.2B --port 8000 --gpu-memory-utilization 0.3
```

1.2B 模型仅 ~3GB，`--gpu-memory-utilization 0.3` 即可。如需和 Ollama 共享 GPU，可降至 0.2。

2.6.3 验证部署

```
1 curl -X POST http://localhost:8000/v1/completions \
2   -H "Content-Type: application/json" \
3   -d '{"model": "opendatalab/MinerU2.5-2509-1.2B", "prompt": "Hello", "max_tokens": 10}'
```

2.6.4 与 MinerU 的区别

维度	MinerU API	vLLM MinerU-1.2B VLM
用途	PDF → Markdown 结构化解析	公式图片 → LaTeX 跨模态对齐
部署位置	宿主机 (Windows)	WSL2 容器
模型尺寸	大模型 (>10B OCR)	1.2B 稠密模型
推理速度	~42s/篇 (PDF 解析)	~1.5s/图 (公式识别)
依赖	MinerU 服务	vLLM + PyTorch CUDA

3. 四路径资源调度器部署

resource_scheduler.py (benchmark/agent/resource_scheduler.py) 是系统的 Layer 2 核心组件。它负责：

- **健康检查**：8 路独立探活，零密钥泄露
- **Provider 参数管理**：从 agent_capabilities.json 加载各 Provider 的模型/端点/温度/超时配置
- **客户端工厂**：每 Provider 独立客户端实例
- **自动降级**：按 resource_type 级联回退

3.1 四路径 Provider 决策

路径	Provider	用途	认证方式
API	DeepSeek API	Intent Router / Judge / 文本生成	OPENAI_API_KEY → Authorization: Bearer
API	MiMo-V2.5 原生视觉	跨模态审计 — 图片→公式语义判决	MIMO_API_KEY → api-key header (raw httpx)
API	MiMo / MiniMax 文本	补充题目生成	MIMO_API_KEY / MINIMAX_API_KEY
Ollama	Ollama 本地	本地 Judge / 本地评测 / --local 模式	无密钥
Ollama Vision	qwen3.5:9b vision	医学图片描述与视觉检索增强	无密钥
vLLM	vLLM WSL2	MinerU-1.2B VLM 公式识别	无密钥 (本机端口)
MinerU	MinerU 本地 API	科学文档 PDF/PPTX/DOCX/XLSX/PNG/扫描件解析	MINERU_API_BASE

3.2 健康检查验证

Agent 启动时自动执行 8 路健康检查：

```
1  conda run --no-capture-output -n myenv python -c "  
2  from benchmark.agent.resource_scheduler import get_global_scheduler  
3  s = get_global_scheduler()  
4  status = s.check_health()  
5  print(status.summary)  
6  "
```

健康检查输出示例：

```
1  ResourceStatus @ 2026-05-10T09:30:00  
2  ✓ deepseek          available    连通正常  
3  ✓ mimo_vision       available    连通正常  
4  ✓ mimo_text         available    连通正常  
5  ✓ minimax           available    连通正常  
6  ✓ ollama_chat        available    连通正常  
7  ✓ ollama_compat      available    连通正常  
8  ✗ mineru             unavailable  MinerU API 未启动  
9  ? gpu               unknown      GPU 检测超时
```

每种 Provider 有独立的超时（5s）和 TTL（30s，缓存避免频繁探活）。

3.3 Provider 参数映射

agent_capabilities.json 的 resource_scheduler_config.provider_parameter_map 定义了各 Provider 的精确调用参数。关键配置：

Provider	子类型	模型	base_url	client_type	特殊说明
deepseek	chat	deepseek-v4-flash	https://api.deepseek.com	openai	proxy=None
mimo	vision	mimo-v2.5	https://api.xiaomimimo.com/v1	raw_httpx	thinking:disabled, api-key header
mimo	text	mimo-v2-flash	https://api.xiaomimimo.com/v1	openai	—
minimax	chat	minimax-2.5	https://api.minimaxi.com/anthropic	anthropic	Anthropic 兼容接口
ollama	chat	qwen3.5:9b	http://localhost:11434	raw_httpx	/api/chat + think:false
ollama	completions	qwen3.5:9b	http://localhost:11434	openai	/v1/chat/completions
vllm	vision	MinerU2.5-2509-1.2B	http://localhost:8000	raw_httpx	WSL2 容器, /v1/completions, 非默认注册

3.4 验证调度器

```
1 # 验证调度器能正确解析注册表中的 15 个 intent 资源需求
2 conda run --no-capture-output -n myenv python -c "
3 from benchmark.agent.resource_scheduler import get_global_scheduler
4 s = get_global_scheduler()
5 for intent_id in ['arxiv_search', 'mineru_parse', 'corpus_parse', 'cross_modal_audit',
6                  'corpus_quality_report', 'benchmark_build', 'quality_arbitrate',
7                  'model_evaluate', 'multi_judge_compare', 'leaderboard_viz',
8                  'local_finetune', 'multi_format_parse', 'medical_rag',
9                  'sciverse_search', 'reproduce_all']:
10     r = s.resolve(intent_id)
11     print(f'{intent_id:25s} → {r.provider:12s} {r.sub_type:10s} {r.model}')
12 "
```

4. 跨模态审计引擎部署

`visual_audit.py` (`benchmark/agent/visual_audit.py`) 是赛道二核心差异化武器。它使用 MiMo-V2.5 原生全模态视觉模型，对语料中的科学图片和 LaTeX 公式进行语义一致性审计。

4.1 核心特性

特性	规格
视觉引擎	MiMo-V2.5 (mimo-v2.5) — 原生全模态, 40 图像 token
兜底引擎	MiniMax (MiniMax-M2.5-highspeed) — 综合对齐质量 79.3%
推理速度	4.7s/图 (MiMo) 、 8.8s/图 (MiniMax)
并发	5 线程 ThreadPoolExecutor
断点续跑	checkpoint 每 50 条原子写入
图片来源	6,204 含图 chunk (23 教材 + 339 arXiv) , 11,554 张图片
审计指标	<code>alignment_pass_rate</code> , <code>per_image_judgment</code> , <code>failure_reasons</code>

4.2 Provider 选择

通过环境变量 `VISION_PROVIDER` 切换：

```
1 # 默认: MiMo-V2.5 (更严格、更快)
2 $env:VISION_PROVIDER = "mimo"
3
4 # 兜底: MiniMax (交叉验证用)
5 $env:VISION_PROVIDER = "minimax"
```


4.3 执行审计

```
1 # 全量扫描（默认 5 线程，断点续跑）
2 conda run --no-capture-output -n myenv python benchmark/agent/visual_audit.py --workers
  5 --resume
3
4 # 限量调试（审计前 100 张图片）
5 conda run --no-capture-output -n myenv python benchmark/agent/visual_audit.py --max-
  items 100
6
7 # 单图测试验证 MiMo 识别能力
8 conda run --no-capture-output -n myenv python benchmark/agent/visual_audit.py --test
```

`--resume` 本身就会自动跳过已完成条目，无需单独的 `--retry-failed` 参数。

4.4 参数说明

参数	默认值	说明
<code>--workers</code>	5	并发线程数（建议 5-8）
<code>--max-items</code>	None（全量）	最大审计图片数（调试用，None=全量）
<code>--resume</code>	false	从 checkpoint 恢复（自动跳过已完成条目）
<code>--test</code>	false	单图测试模式
<code>--provider</code>	环境变量 <code>VISION_PROVIDER</code>	覆盖视觉 Provider: <code>mimo</code> 或 <code>minimax</code>

4.5 审计产出

文件	格式	说明
<code>visual_audit_results.jsonl</code>	JSONL	逐条审计结果（图片引用+对应公式+语义判决）
<code>visual_audit_checkpoint.json</code>	JSON	断点续跑 checkpoint
<code>visual_audit_report.json</code>	JSON	汇总统计报告（通过率/失败原因分布）

4.6 与 Intent Router 集成

跨模态审计在 `agent_cli.py` 中通过 intent `cross_modal_audit` 自动调度：

```
1 # 自然语言触发
2 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
3     "审计这批 arXiv 论文的公式-图片跨模态对齐质量"
4
5 # 手动指定 intent 序列
6 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
7     --intents corpus_parse,cross_modal_audit,corpus_quality_report
```

5. Agent CLI — 15 intents × LLM Intent Router

`agent_cli.py`（`benchmark/agent/agent_cli.py`）是系统的 Layer 1+3 核心入口。它将用户的自然语言请求通过 LLM few-shot 拆解为可执行的 intent 序列，然后逐步执行并验证。

5.1 基本用法

```
1 # 自然语言驱动 (DeepSeek v4-flash 自动拆解)
2 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
3     "全量复现整个评测流程"
4
5 # 指定 intent 序列 (跳过 Intent Router)
6 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
7     --intents
8     corpus_parse,benchmark_build,quality_arbitrate,model_evaluate,leaderboard_viz
9
10 # dry-run 验证计划 (不实际执行)
11 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
12     --dry-run "评测 deepseek-v4-flash 模型"
```

5.2 完整参数

参数	默认值	说明
<code>input</code>	无 (必填之一)	自然语言任务描述
<code>--intents / -i</code>	无	手动指定 intent 序列，逗号分隔
<code>--dry-run / -n</code>	false	干跑模式：仅生成计划不执行
<code>--local / -l</code>	false	本地模式：Resource Scheduler 降级至 Ollama/MinerU (Intent Router 仍用 DeepSeek)
<code>--max-retries</code>	2	每步最大重试次数
<code>--no-verify</code>	false	禁用 LLM 验证
<code>--output-log / -o</code>	<code>logs/{task_id}.json</code>	执行日志 JSON 输出路径
<code>--list-intents</code>	false	列出所有可用 intent
<code>--version / -v</code>	false	显示版本号

5.3 本地降级模式 (--local)

`--local` 模式是赛道二的隐私双路径核心特性。启用后 Resource Scheduler 将 API 类 intent 的 Provider 降级至本地：

```
1 # 本地模式: Resource Scheduler 降级至 Ollama (Intent Router 仍用 DeepSeek)
2 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
3     --local "生成语料质量报告"
4
5 # 本地模式指定 intent
6 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
7     --local --intents corpus_parse,corpus_quality_report
```

`--local` 模式下，Resource Scheduler 将 API 类 intent 的 provider 降级至 Ollama 或 script，详情见下表：

原路径	<code>--local</code> 降级至	说明
DeepSeek API	Ollama qwen3.5:9b	<code>_resolve_local</code> 将各 intent 映射至 <code>ollama</code>
MiMo-V2.5 视觉	由 <code>VISION_PROVIDER</code> 控制	resolve 降级至 <code>ollama</code> ，但 Executor 执行 <code>visual_audit.py</code> 时视觉引擎由 <code>VISION_PROVIDER</code> 环境变量决定（默认 MiMo）
MiniMax API	Ollama qwen3.5:9b	同上
MinerU API	本地 MinerU	<code>corpus_parse</code> 映射至 <code>mineru</code>
vLLM WSL2	不可降级（WSL2 内无备选）	vLLM 仅用于公式识别对比实验，非核心路径

注意：`--local` 仅影响 Resource Scheduler 的 resolve 策略。**Intent Router (Layer 1)** 始终调用 **DeepSeek v4-flash**，不受 `local_mode` 影响。

5.4 Intent 序列示例

intent 0 — arXiv 论文检索（数据飞轮起点）

```
1 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2     --intents arxiv_search
```

intent 1 — 文档解析（原始 PDF→Markdown）

```
1 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2     --intents mineru_parse
```

intent 2 — 语料解析

```
1 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2     --intents corpus_parse
```

intent 3 — 跨模态审计

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents cross_modal_audit
```

intent 4 — 语料质量报告

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents corpus_quality_report
```

intent 5 — 评测数据集构建

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents benchmark_build
```

intent 6 — 质量仲裁

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents quality_arbitrate
```

intent 7 — 模型评测

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents model_evaluate
```

intent 8 — 多 Judge 对比

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents multi_judge_compare
```

intent 9 — 排行榜可视化

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents leaderboard_viz
```

intent 10 — 本地微调

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents local_finetune
```

intent 11 — 多格式文档解析 (PPTX/DOCX/XLSX/PNG/扫描件)

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents multi_format_parse
```

intent 12 — 医学文献 RAG

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents medical_rag
```

intent 13 — Sciverse 文献检索

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents sciverse_search
```

intent 14 — 全量复现

```
1 | conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
2 |     --intents reproduce_all
```

5.5 执行日志

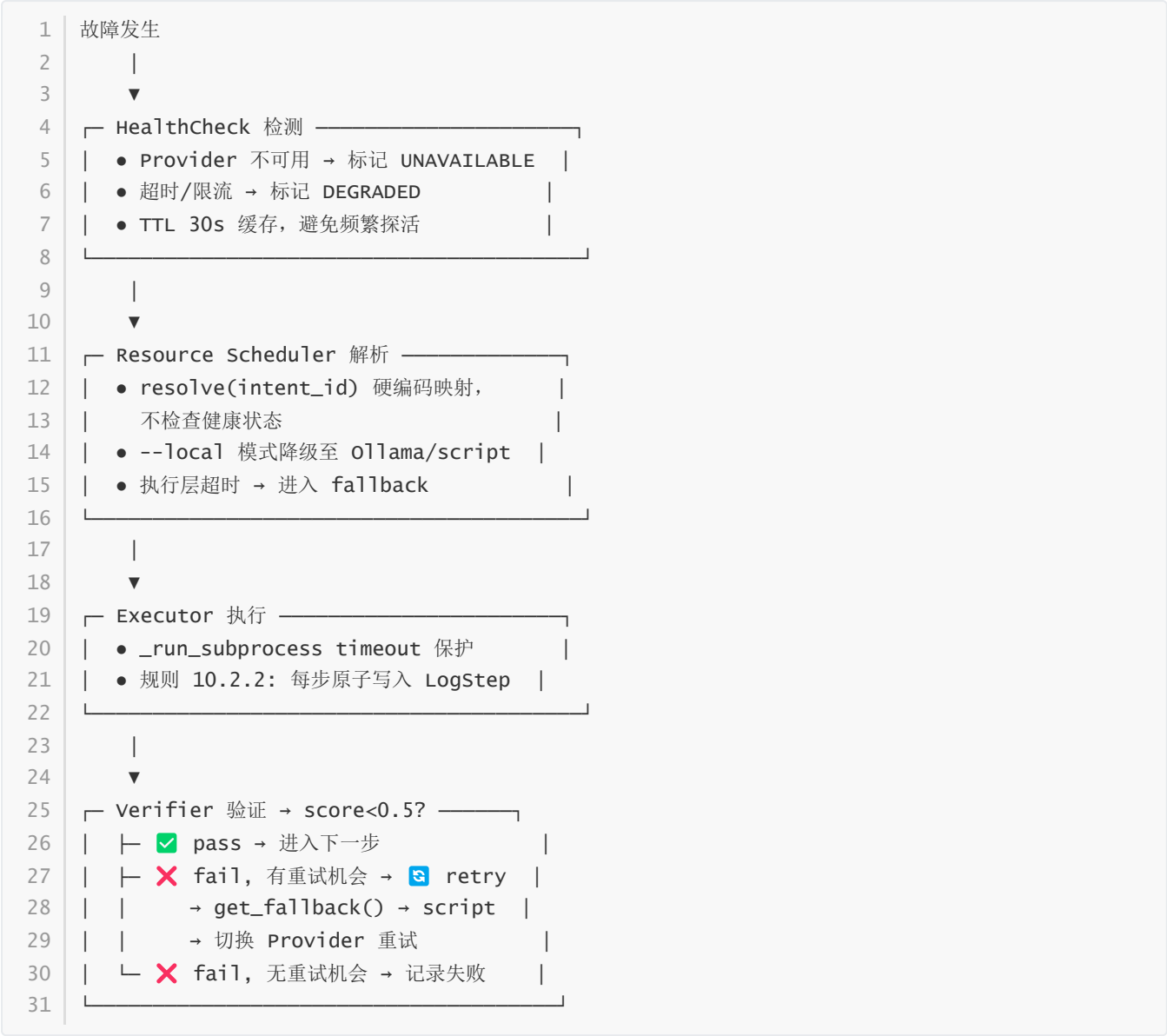
Agent 每次执行自动生成结构化 JSON 日志，保存至 `benchmark/agent/logs/`（可通过 `--output-log` 自定义路径）：

```
1 | {
2 |   "schema_version": "1.0",
3 |   "task_id": "agent-20260508-153000",
4 |   "task_name": "审计这批 arxiv 论文的跨模态对齐质量",
5 |   "steps": [
6 |     {
7 |       "schema_version": "1.0",
8 |       "step_id": 1,
9 |       "step_name": "cross_modal_audit (跨模态对齐审计)",
10 |      "tool": "visual_audit.py + MiMo-v2.5 原生视觉",
11 |      "input_summary": "{\"source\": \"corpus/chunks/\", \"concurrency\": 5,
12 |      \"resume\": true}",
13 |      "output_summary": "✅ Audit complete: 4935 images, 79.3% alignment pass rate",
14 |      "status": "success",
15 |      "duration_ms": 1080000,
16 |      "timestamp": "2026-05-08 15:30:45",
17 |      "error": null
18 |    },
19 |    "total_duration_ms": 1080000,
20 |    "final_status": "completed",
21 |    "created_at": "2026-05-08 15:12:45"
22 |  }
```

6. 故障恢复矩阵

系统设计了四层容错机制，覆盖 Provider 不可用、API 超时、JSON 解析失败、Ollama 熔断等常见故障。

6.1 故障恢复路线图



6.2 Provider 级降级

重要说明： `resolve()` 是硬编码映射，不检查健康状态。真正的 fallback 发生在 **Executor 层超时后**，由 `get_fallback()` 执行。所有 Provider 的 fallback 目标均为 `script`（本地脚本），不是另一个 Provider。

故障模式	检测层	降级策略（代码实际行为）
DeepSeek API 执行超时	Executor <code>_run_subprocess</code>	<code>get_fallback()</code> → script（本 intent 以本地脚本执行）
MiMo-V2.5 视觉 404	Executor 执行报错	<code>get_fallback()</code> → script（本 intent 以本地脚本执行）
MiniMax 限流 429	Executor 执行报错	<code>get_fallback()</code> → script
Ollama 服务未启动	Executor 执行报错	<code>get_fallback()</code> → script
MinerU API 未监听	Executor 执行报错	<code>get_fallback()</code> → script

故障模式	检测层	降级策略（代码实际行为）
vLLM WSL2 未启动	Executor 执行报错	无 fallback（对比实验可选，跳过不影响核心链路）
Ollama Judge 熔断	<code>--max-consecutive-failures</code> 阈值	API Judge = 5, Ollama Judge ≥ 10
JSON 解析失败	<code>_extract_json_block()</code> 两层回退	修复 JSON (尾逗号清理) → 重试

6.3 Resource Type 级降级

定义于 `agent_capabilities.json` 的 `resource_scheduler_config.fallback_order`：

资源类型	允许降级至	超时
api	script	120s
local_api	script	300s
local_gpu	script	3600s
script	不降级	600s

6.4 重试 + 验证闭环

每步执行自动经历 `Execute` → `LLM Verify` → `Auto-fallback` 闭环：

1

Step 失败 — 首次自动重试（provider 不变）

2

├ 成功 →  继续

3

└ 失败 →  二次重试（降级 provider）

4

├ 成功 →  继续

5

└ 失败 →  记录失败，继续下一步

- 最大重试次数：`--max-retries 2`（默认）
- Verify 失败判定：`score < 0.5`
- 降级检测：`scheduler.get_fallback(resolved)` 返回备选 Provider

6.5 隐私双路径

模式	API 依赖	适用场景
默认	DeepSeek + MiMo + MinerU + vLLM（可选）	全功能模式
<code>--local</code>	仍有 DeepSeek（Intent Router）	减少 API 调用量，适合轻度规划场景

`--local` 模式下：

- **Intent Router（Layer 1）始终调用 DeepSeek**，不受 `local_mode` 影响

- Resource Scheduler (Layer 2) 将 API 类 intent 降级至 Ollama (`_resolve_local` 映射)
- `cross_modal_audit` 映射至 `ollama` , 但 Executor 仍执行 `visual_audit.py` (视觉由 MiMo/MiniMax 提供)
- `--local` 不等于零 API, 仅减少了执行层的外部 API 依赖

6.6 断点续跑

系统在多个层面支持断点续跑:

组件	恢复机制	粒度
visual_audit.py	<code>--resume</code> + checkpoint (每 50 条)	逐条
evaluate.py	<code>--resume</code> + <code>--retry-failed</code>	逐题
build_benchmark.py	<code>--resume</code> + checkpoint (每 15 题)	逐题
agent_cli.py	重新运行跳过已完成步骤	按 step
reproduce_all	<code>skip_existing=True</code> 检测已有产出	按 intent

7. 语料准备

7.1 语料来源

ControlSci 语料库包含两类来源:

来源	数量	格式	存储位置
控制科学教材	23 本	PDF	<code>data/pdf/textbooks/</code>
arXiv 论文	339 篇	PDF	<code>data/pdf/arxiv/</code>

7.2 语料解析 (MinerU)

对应 intent `corpus_parse` , 或直接使用 MinerU 命令行:

```
1 # 批量解析教材
2 conda run --no-capture-output -n myenv magic-pdf -p data/pdf/textbooks/ -o
  corpus/parsed/textbooks/ -m auto
3
4 # 批量解析 arXiv 论文
5 conda run --no-capture-output -n myenv magic-pdf -p data/pdf/arxiv/ -o
  corpus/parsed/arxiv/ -m auto
```

性能参考: arXiv 论文平均解析速度约 42 秒/篇 (62% 固定开销) , 教材约 60 页/分钟。

7.3 多格式解析测试素材

对应 intent `multi_format_parse`，需准备以下测试文件：

格式	建议来源	存储位置
PPTX	控制科学课件（教材配套幻灯片）	<code>benchmark/agent/test_materials/</code>
DOCX	技术说明或实验记录文档	<code>benchmark/agent/test_materials/</code>
XLSX	结构化表格或评测结果表	<code>benchmark/agent/test_materials/</code>
PNG / 扫描件图片	公式密集页（教材截图）	<code>benchmark/agent/test_materials/</code>

```
1 # 多格式解析：指定文件列表
2 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py `
3     --intents multi_format_parse
4
5 # 直接调用 mineru_to_md.py（跳过 Agent 编排）
6 conda run --no-capture-output -n myenv python tools/mineru_to_md.py `
7     --input benchmark/agent/test_materials/ --output benchmark/agent/test_materials/md/
8     --skip-existing --stats
```

产出的 `parse_result_*.md` 文件按 chunk 数/公式提取率/图片保留率三维度对比各格式的解析质量差异。

8. 题目生成

8.1 三 Provider 并行生成

对应 intent `benchmark_build`，或直接使用 `build_benchmark.py`：

```
1 # DeepSeek 生成（默认）
2 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py `
3     --corpus corpus --provider deepseek --limit 50 `
4     --output benchmark/dataset/benchmark.json `
5     --manual-review-output benchmark/dataset/manual_review.json
6
7 # MiniMax 生成
8 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py `
9     --corpus corpus --provider minimax --limit 50 `
10    --output benchmark/dataset/benchmark_minimax.json `
11    --manual-review-output benchmark/dataset/manual_review_minimax.json
12
13 # MiMo 生成
14 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py `
15    --corpus corpus --provider mimo --limit 50 `
16    --output benchmark/dataset/benchmark_mimo.json `
17    --manual-review-output benchmark/dataset/manual_review_mimo.json
```

8.2 关键参数

参数	默认值	说明
<code>--corpus</code>	<code>corpus/</code>	语料库根目录
<code>--provider</code>	<code>deepseek</code>	Provider: <code>deepseek</code> / <code>minimax</code> / <code>mimo</code>
<code>--limit</code>	10	生成题目数量
<code>--output</code>	<code>benchmark/dataset/benchmark.json</code>	输出 JSON 路径
<code>--resume</code>	false	从 checkpoint 恢复
<code>--supplement</code>	0	追加生成 N 题到已有 <code>benchmark.json</code>
<code>--reset-every</code>	15	每 N 题重建 OpenAI 客户端

8.3 Checkpoint 机制

生成过程自动保存 checkpoint 到 `benchmark/archive/checkpoint/`。若生成中断：

```
1 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py `
2   --corpus corpus --provider deepseek --limit 50 --resume `
3   --output benchmark/dataset/benchmark.json `
4   --manual-review-output benchmark/dataset/manual_review.json
```

9. 质量仲裁

9.1 双层 LLM 仲裁

对应 intent `quality_arbitrate`，使用 DeepSeek 进行 Embedding 快速通道 + LLM 深度仲裁：

```
1 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py `
2   --arbitrate --output benchmark/dataset/benchmark.json `
3   --manual-review-output benchmark/dataset/manual_review.json
```

9.2 仲裁参数

参数	默认值	说明
<code>--arbitrate-workers</code>	3	并行仲裁 API 调用数
<code>--arbitrate-max-tokens</code>	1024	单次仲裁响应最大 token 数

9.3 仲裁状态流转

```
1 生成 → needs_review → [Embedding 快速通道] → auto_passed
2                               ↳ [LLM 深度仲裁] → reviewed_kept / reviewed_discarded
```

9.4 孤儿回收

```
1 conda run --no-capture-output -n myenv python benchmark/pipeline/orphan_judge.py
```

孤儿回收使用 DeepSeek API，单轮调用，185 题约 9 分钟完成，通过率约 61.6%。

10. 数据集构建 + 评测 + 报告

10.1 数据集拆分

```
1 conda run --no-capture-output -n myenv python benchmark/pipeline/split_benchmark.py `
2 --input benchmark/dataset/merged.json --seed 42
```

文件	题数	说明
benchmark/dataset/core.json	500	核心集: A/B/C/D 各 125
benchmark/dataset/full.json	889	全量可用集
benchmark/dataset/merged.json	1,109	候选池

10.2 模型评测

对应 intent `model_evaluate`，支持 API / Ollama 双模式：

```
1 # API 模型评测 (DeepSeek)
2 conda run --no-capture-output -n myenv python benchmark/eval/evaluate.py `
3 --mode model --input benchmark/dataset/core.json `
4 --target-model deepseek-v4-flash --judge-model deepseek-v4-flash `
5 --base-url https://api.deepseek.com `
6 --output benchmark/eval/reports/deepseek-v4-flash_report.json `
7 --resume --retries 3 --max-consecutive-failures 5
8
9 # 本地模型评测 (ollama qwen3.5:9b)
10 conda run --no-capture-output -n myenv python benchmark/eval/evaluate.py `
11 --mode model --input benchmark/dataset/core.json `
12 --target-model qwen3.5:9b --target-base-url http://localhost:11434/v1 `
13 --target-api-key ollama `
14 --judge-model deepseek-v4-flash --judge-base-url https://api.deepseek.com `
15 --output benchmark/eval/reports/qwen3.5-9b_report.json `
16 --resume --retries 3 --max-consecutive-failures 10
```

关键：Ollama Judge 的 `--max-consecutive-failures` 必须设为 **10** 而非默认 5（规则 10.8.3）。C/D 维低分易被误判为失败。

10.3 排行榜 + 可视化

对应 intent `leaderboard_viz`：

```
1 # 生成排行榜 JSON
2 conda run --no-capture-output -n myenv python benchmark/eval/summarize_multi.py `
3   --input benchmark/eval/reports --output benchmark/eval/reports/leaderboard.json
4
5 # 生成可视化 HTML + PNG
6 conda run --no-capture-output -n myenv python benchmark/eval/report_viz.py `
7   --input benchmark/eval/reports/leaderboard.json `
8   --output benchmark/eval/reports/leaderboard.html
```

10.4 HuggingFace 导出

```
1 conda run --no-capture-output -n myenv python benchmark/pipeline/export_dataset.py `
2   --input benchmark/dataset/core.json --output huggingface/control-sci-corpus
3
4 # 推送到 HuggingFace Hub
5 conda run --no-capture-output -n myenv python benchmark/pipeline/export_dataset.py `
6   --input benchmark/dataset/core.json --output huggingface/control-sci-corpus `
7   --push-to-hub "<HF_USERNAME>/control-sci-benchmark" --hf-token "<HF_TOKEN>"
```

11. 目录结构

```
1 MinerU/
2 |─ benchmark/
3 | |─ agent/ # Data Agent 核心层
4 | | |─ __init__.py
5 | | |─ agent_cli.py # Layer 1+3: Intent Router + Executor + Verifier
6 | | |─ agent_capabilities.json # 15 intents 能力注册表
7 | | |─ resource_scheduler.py # Layer 2: 四路径资源调度器
8 | | |─ visual_audit.py # 跨模态审计引擎（MiMo-V2.5 原生视觉）
9 | | |─ log_schema.py # 日志 Schema 定义
10 | | |─ agent.py # 原 v1 ControlSciAgent（向后兼容）
11 | | |─ _validate_capabilities.py # capabilities.json 校验
12 | | |─ _verify_10_intents.py # intent dry-run 验收脚本（历史文件名保留）
13 | | |─ test_materials/ # 多格式解析测试素材（PPTX/DOCX/XLSX/PNG/图片）
14 | | |─ logs/ # 执行日志 JSON
15 | | |─ results/ # visual_audit 审计产出
16 | | |─ examples/ # 12 个示例脚本
17 | | | |─ run_task_1_corpus.py
18 | | | |─ run_task_2_arbitrate.py
19 | | | |─ run_task_3_eval.py
20 | | | |─ run_task_4_recovery.py
21 | | | |─ run_task_5_transfer.py
22 | | |─ logs/ # 示例执行日志
```

```

23 | | └─ dataset/                # 数据集 JSON
24 | |   └─ core.json            # 核心集（500 题）
25 | |   └─ full.json           # 全量集（889 题）
26 | |   └─ schema.json         # 数据 Schema
27 | └─ pipeline/               # 数据生成管线
28 | |   └─ build_benchmark.py   # 题目生成 + 仲裁入口
29 | |   └─ split_benchmark.py   # core/full 拆分
30 | |   └─ merge_benchmarks.py  # 多路合并
31 | |   └─ orphan_judge.py      # 孤儿题回收
32 | |   └─ validate_benchmark.py # 数据校验
33 | |   └─ export_dataset.py    # HuggingFace 导出
34 | └─ eval/                   # 评测引擎
35 |   └─ evaluate.py           # 评测入口
36 |   └─ judge.py              # LLM-as-Judge 评分器
37 |   └─ summarize_multi.py     # 多模型排行榜
38 |   └─ report_viz.py          # 可视化
39 |   └─ reports/               # 评测报告输出
40 └─ corpus/                   # 语料解析产物（chunk、元数据、统计）
41 └─ data/                     # 原始数据源
42 |   └─ pdf/textbooks/        # 23 本控制科学教材 PDF
43 |   └─ pdf/arxiv/            # 339 篇 arXiv 论文 PDF
44 |   └─ sources_flywheel/     # 数据飞轮论文 PDF
45 └─ tools/                    # 辅助工具
46 |   └─ mineru_to_md.py       # MinerU 解析封装（多格式支持）
47 └─ docs/
48 |   └─ agent/
49 |     └─ deploy.md           # 本文档
50 |     └─ agent_report.md     # 技术报告
51 └─ finetune/                 # QLORA 微调模块
52 |   └─ train_qlora.py
53 |   └─ data/                 # 微调数据集

```

12. 快速验证清单

完成部署后，按以下顺序验证各组件是否正常工作：

```

1  # 1. 环境检查
2  conda run --no-capture-output -n myenv python -c "import openai; import anthropic;
import httpx; print('deps OK')"
3
4  # 2. Provider 健康检查（8 路）
5  conda run --no-capture-output -n myenv python -c "
6  from benchmark.agent.resource_scheduler import get_global_scheduler
7  s = get_global_scheduler()
8  print(s.check_health().summary)
9  "
10
11 # 3. Agent CLI 模块 import
12 conda run --no-capture-output -n myenv python -c "
13 from benchmark.agent.agent_cli import ControlSciAgentCLI, IntentRegistry, IntentRouter,
Executor, Verifier

```

```

14 print('agent OK')
15 "
16
17 # 4. 15 intents 能力注册表验证
18 conda run --no-capture-output -n myenv python benchmark/agent/_validate_capabilities.py
19
20 # 5. intent dry-run 验收
21 conda run --no-capture-output -n myenv python benchmark/agent/_verify_10_intents.py
22
23 # 6. agent_cli.py dry-run (自然语言)
24 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py --dry-run
    "审计跨模态对齐质量"
25
26 # 7. agent_cli.py dry-run (--local 模式)
27 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py --dry-run --
    local "生成语料质量报告"
28
29 # 8. visual_audit 单图测试
30 conda run --no-capture-output -n myenv python benchmark/agent/visual_audit.py --test
31
32 # 9. 数据校验
33 conda run --no-capture-output -n myenv python benchmark/pipeline/validate_benchmark.py
    --input benchmark/dataset/core.json
34
35 # 10. Reference 评测测试
36 conda run --no-capture-output -n myenv python benchmark/eval/evaluate.py --mode
    reference --input benchmark/dataset/core.json --output test_ref.json
37 Remove-Item test_ref.json -ErrorAction SilentlyContinue
38
39 # 11. Mock 生成测试
40 conda run --no-capture-output -n myenv python benchmark/pipeline/build_benchmark.py --
    mock --limit 3 --output test_mock.json --manual-review-output test_review.json
41 Remove-Item test_mock.json, test_review.json -ErrorAction SilentlyContinue
42
43 # 12. (可选) vLLM WSL2 独立连通性 (非 HealthCheck 内置, 需单独验证)
44 conda run --no-capture-output -n myenv python -c "
45 import httpx
46 r = httpx.get('http://localhost:8000/v1/models', timeout=5)
47 print(f'vLLM: {r.status_code}')
48 "
49
50 # 13. (可选) 多格式解析 dry-run
51 conda run --no-capture-output -n myenv python benchmark/agent/agent_cli.py --dry-run
    "把 PPTX 课件转成结构化文本"

```

全部通过即部署成功。

13. 常见问题排查

13.1 MiMo-V2.5 视觉 API 返回 404

症状：visual_audit.py 返回 HTTP 404。

原因：使用了 `mimo-v2.5-pro` 模型（长推理模型，不支持图像输入）。视觉模型必须是 `mimo-v2.5`（原生全模态）。

解决：

- 确认 `agent_capabilities.json` 中 `mimo.vision.model` 为 `mimo-v2.5`
- MiMo 视觉调用必须使用 **raw httpx** + `api-key` header，严禁 OpenAI SDK
- 必须设置 `thinking:disabled`

13.2 Ollama Judge 熔断误杀

症状：Ollama Judge 评测到一半进程退出，日志显示 "max consecutive failures reached"。

原因：C/D 维低分被误判为失败，默认 `--max-consecutive-failures 5` 过激。

解决：Ollama Judge 必须使用 `--max-consecutive-failures 10`，与 API Judge (5) 区分。

13.3 Ollama 推理链占用过多 token

症状：Qwen 3.5 评测时 `content` 字段为空，`max_tokens` 被推理链耗尽。

解决：

- `/api/chat`（原生，推荐）：设置 `think:false`
- `/v1/chat/completions`（兼容）：设置 `reasoning_effort='none'`

13.4 Import 路径错误

症状：`ModuleNotFoundError: No module named 'benchmark'`。

解决：确保从项目根目录运行脚本。`:file_path` 模块已在 `agent_cli.py` 和 `resource_scheduler.py` 中自动处理：

```
1 ROOT = Path(__file__).resolve().parents[2]
2 if str(ROOT) not in sys.path:
3     sys.path.insert(0, str(ROOT))
```

13.5 Conda 编码崩溃

症状：`UnicodeDecodeError: 'gbk' codec can't decode byte`。

解决：所有子进程调用必须使用：

```
1 $env:PYTHONIOENCODING='utf-8'
2 $env:CONDA_NO_PLUGINS='true'
3 conda run --no-capture-output -n myenv python ...
```

13.6 MiniMax 限流

症状：MiniMax API 返回 429 错误。

解决：MiniMax 限流阈值约 40 题/路。使用 `--limit` 分批生成，或降低 `--arbitrate-workers`。

13.7 DeepSeek v4 系列 JSON 截断

症状：Intent Router 返回的 JSON 计划不完整（被截断）。

解决：中文 JSON 输出场景 `max_tokens` 至少 4096（当前默认值）。`agent_cli.py` 和 `resource_scheduler.py` 中已统一设置为 4096。

13.8 Checkpoint 文件损坏

症状：`--resume` 后数据异常。

解决：删除对应 checkpoint 文件后重新运行：

```
1 Remove-Item benchmark/archive/checkpoint/*.checkpoint.json
2 Remove-Item benchmark/agent/results/visual_audit_checkpoint.json
```

13.9 vLLM WSL2 连通失败

症状：vLLM 调用超时或端口不可达。

原因：WSL2 未启动 / vLLM 服务未运行 / 端口冲突。

解决：

1. 确认 WSL2 已启动：`wsl --list --verbose`
2. WSL 内确认 vLLM 进程运行：`ps aux | grep vllm`
3. 检查端口占用：WSL 内 `ss -tlnp | grep 8000`
4. 宿主机 → WSL 端口转发：`netsh interface portproxy show v4tov4`
5. 如端口被占，改用 `--port 8001` 并更新 `agent_capabilities.json` 的 `vllm.base_url`

vLLM 是可选组件（仅用于 MinerU-1.2B VLM 公式识别对比实验），不影响核心评测链路。启动失败时跳过即可。